



AWS Whitepaper

Build Modern Data Streaming Architectures on AWS



Build Modern Data Streaming Architectures on AWS: AWS Whitepaper

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

.....	v
Abstract and introduction	i
Abstract	1
Are you Well-Architected?	1
Introduction	2
What is a modern data architecture?	3
What is a modern data streaming architecture?	6
Working with streaming data on AWS	9
Amazon Kinesis Data Streams	11
Amazon Data Firehose	13
Amazon Managed Service for Apache Flink	14
Amazon Kinesis Video Streams	15
Amazon MSK	15
Streaming architecture patterns using a modern data architecture	18
Low latency modern data streaming applications	18
Build access logs streaming applications using Firehose and Managed Service for Apache Flink.	18
Stream data from diverse source systems into the data lake using MSK for near real-time reports	19
Build a serverless streaming data pipeline using Amazon Kinesis and AWS Glue	20
Set up near real-time search on DynamoDB table using Kinesis Data Streams and OpenSearch Service	22
Build a real-time fraud prevention system using Amazon MSK and Amazon Fraud Detector	23
Stream games data from diverse source systems into a Lake using Kinesis Data Streams for real-time game insights	25
Key considerations while building streaming analytics	27
Choosing the right Kinesis service for your use case	27
Choosing the right streaming for your use case	28
Choosing the right streaming data processing technology	29
Key benefits	34
Conclusion	35
Contributors	36
Further reading	37

Document revisions

Notices

AWS Glossary

38

39

40

This whitepaper is for historical reference only. Some content might be outdated and some links might not be available.

Build Modern Data Streaming Architectures on AWS

Publication date: **May 17, 2022** ([Document revisions](#))

Abstract

Modern data architecture is about using the right tool for the job. It acknowledges that a “one size fits all” approach leads to compromise, and a solution that is not optimized for anyone. With modern data architecture, customers can integrate a data lake, data warehouses, and purpose-built data services, with a unified governance layer, to create a system without boundaries that enables data-driven decisions.

When building a modern data architecture, sometimes there is a need for data to flow with low latency between components to power real-time decisions. This whitepaper helps cloud architects, data scientists, and developers to design and building modern data streaming architectures that can quickly generate insights using Amazon Web Services (AWS) streaming services such [Amazon Kinesis Data Streams](#), [Amazon Data Firehose](#), [Amazon Managed Service for Apache Flink](#), and [Amazon Managed Streaming for Apache Kafka](#) (Amazon MSK).

Are you Well-Architected?

The [AWS Well-Architected Framework](#) helps you understand the pros and cons of the decisions you make when building systems in the cloud. The six pillars of the Framework allow you to learn architectural best practices for designing and operating reliable, secure, efficient, cost-effective, and sustainable systems. Using the [AWS Well-Architected Tool](#), available at no charge in the [AWS Management Console](#) (sign-in required), you can review your workloads against these best practices by answering a set of questions for each pillar.

In the [Data Analytics Lens](#), we focus on how to design, deploy, and architect your data analytics workloads in the AWS Cloud. This lens adds to the best practices described in the Well-Architected Framework.

For more expert guidance and best practices for your cloud architecture—reference architecture deployments, diagrams, and whitepapers—refer to the [AWS Architecture Center](#).

Introduction

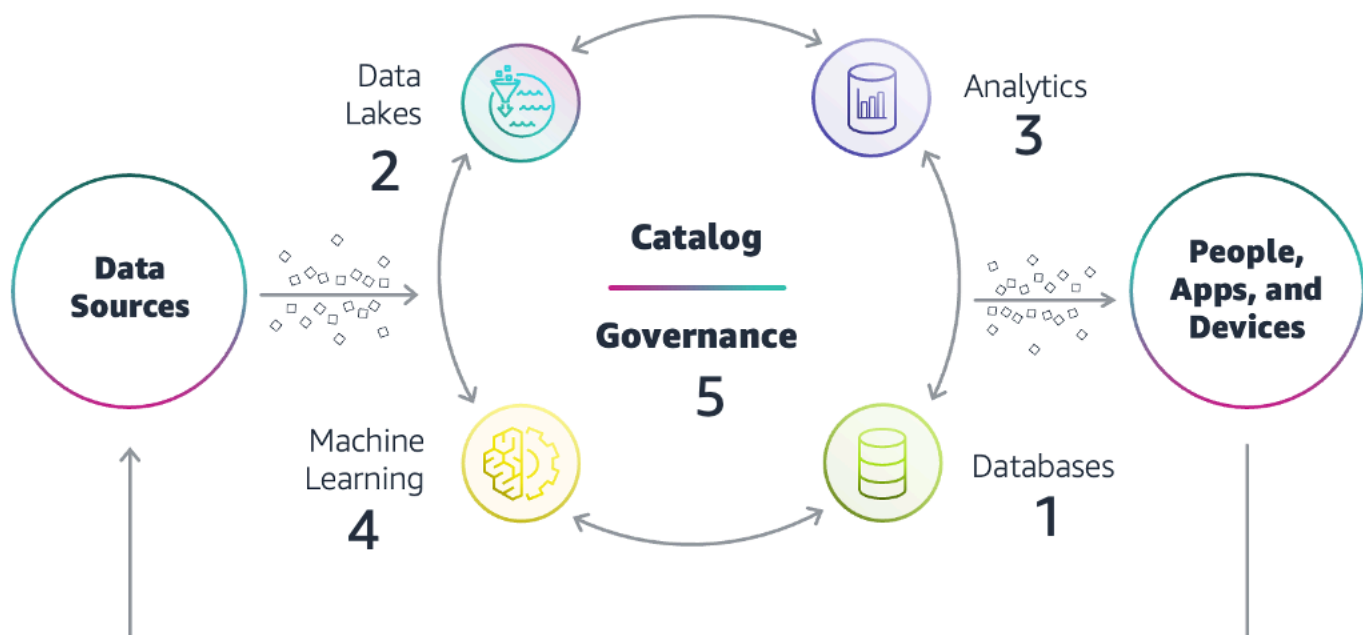
Traditional on-premises data analytics approaches can't handle exponential data volumes because they don't scale well enough and are too expensive. Organizations need to easily access and analyze all types of data, such as structured, semi-structured, unstructured, and real-time streaming data to perform comprehensive and efficient analytics. They also need to easily break down data silos to gain new insights and build better experiences. With a modern data architecture, organizations can collect, store, organize, and process valuable data, make it available in a secure way, and enable applications to derive low-latency, near real-time insights.

This whitepaper presents how you can implement a [modern data architecture on AWS](#) and realize the benefits of low latency insights with streaming technologies. This modern data architecture enables you to collect, manage, process, and analyze all your real-time streaming data in a simple and integrated fashion. A modern data streaming architecture also allows you to use all your data for a variety of use cases, such as streaming logs and event data to build live dashboards, delivering streaming data into data lakes and data warehouses, and building real-time analytics and event driven applications.

We first discuss the concept of the modern data architecture approach, modern data streaming architecture, then present three modern data architecture data movement patterns to derive insights from your near real-time streaming data, using AWS purpose-built analytics services.

What is a modern data architecture?

[A modern data architecture on AWS](#) allows you to build a scalable data lake, and use a broad and deep collection of purpose-built data services that provide the performance required for use cases such as low latency streaming analytics, interactive dashboards, log analytics, big data processing, and data warehousing. It enables you to easily move data between the data lake and purpose-built data services. It also helps you set up governance and compliance in a unified way to secure, monitor, and manage access to your data. AWS calls this modern, cloud-based analytics architecture the *modern data architecture*, as depicted in the following figure.



Modern data architecture on AWS

The modern data architecture includes the following key components:

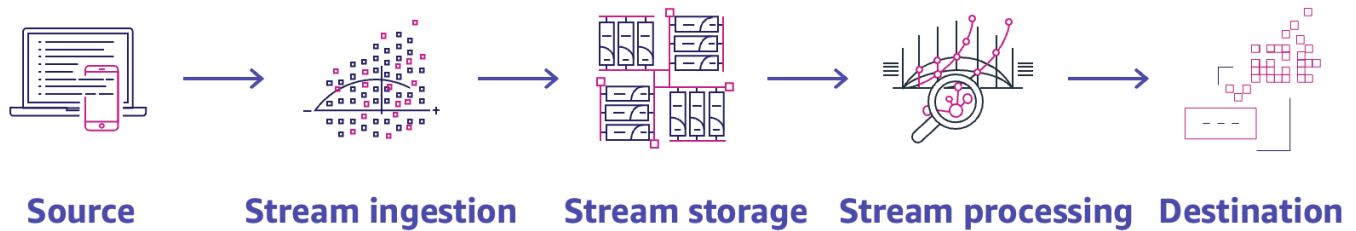
- **Databases** – You can store data in purpose-built databases that can support a modern application and its different features. This database no longer needs to be a single type of relational database—it could be a NoSQL database, cache store, or anything else that works for the application. AWS offers over 15 [purpose-built engines](#) to support diverse data models, including relational, key-value, document, in-memory, graph, time series, wide column, and ledger databases.

- **Data lakes** – It makes sense to then use a data lake on a storage service like [Amazon S3](#) so you can store data from all those purposes-built databases, preferably in its native format or in an open file format. AWS-powered [data lakes](#), supported by the unmatched availability of Amazon S3, can handle the scale, agility, and flexibility required to combine different data and analytics approaches.
- **Analytics** – Once the data lake is hydrated with data, you can easily build modern analytics, which might range from traditional data warehousing and batch reporting to more real-time analytics, near real-time alerting and reporting, and so on. It might even be one-time querying of data or more advanced ML-based analytics use cases. Organizations aren't constrained within data silos because data is now stored in a layer that is more open and allows more freedom for performing comprehensive analytics. AWS provides the broadest and deepest portfolio of [purpose-built analytics services](#), including [Athena](#), [Amazon EMR](#), [OpenSearch Service](#), [Kinesis](#), [Amazon MSK](#), and [Amazon Redshift](#) for your unique analytics use cases.
- **Machine learning** – ML and AI are also critical for a modern data strategy to help organizations predict what might happen in the future and build intelligence into their systems and applications. AWS offers the broadest and deepest set of [machine learning services](#) and supporting cloud [infrastructure](#), putting machine learning in the hands of every developer, data scientist, and expert practitioner. When you build an ML-based workload in AWS, you can choose from three different levels of ML services to balance speed-to-market with your level of customization and ML skill level: [AI services](#), [ML services](#), and [ML frameworks and infrastructure](#).
- **Data governance** – Finally, data governance is a critical element to combining and sharing data from different sources so you can put data in the hands of people at all levels of your organization. Metadata is an integral part of data management and governance. The [AWS Glue Data Catalog](#) can provide a uniform repository to store and share metadata. The main purpose of the Data Catalog is to provide a central metadata store where disparate systems can store, discover, and use that metadata to query and process the data. Another important aspect of data governance is serving and managing the relationship between data stores and external clients, which are the producers and consumers of data. As the data evolves, especially in streaming use cases, we need a central framework that provides a contract between producers and consumers to enable schema evolution and improved governance. The [AWS Glue Data Catalog](#) provides a centralized framework to help manage and enforce schemas on data streaming applications using convenient integrations with Apache Kafka and [Amazon Managed Streaming for Apache Kafka](#), [Amazon Kinesis Data Streams](#), [Apache Flink](#), [Amazon Managed Service for Apache Flink](#), and [Lambda](#).

AWS provides a broad platform of managed services to help you build, secure, and seamlessly scale end-to-end data analytics applications quickly, using the modern data architecture approach. There is no hardware to procure, no infrastructure to maintain and scale—only what you need to collect, store, process, and analyze your data. AWS offers analytical solutions specifically designed to handle this growing amount of data, and provide insight into your business.

What is a modern data streaming architecture?

A modern data streaming architecture allows you to ingest, process, and analyze high volumes of high-velocity data from a variety of sources in real-time to build more reactive and intelligent customer experiences. The modern streaming data architecture can be designed as a stack of five logical layers; each layer is composed of multiple purpose-built components that address specific requirements. The following diagram illustrates the modern streaming data architecture.

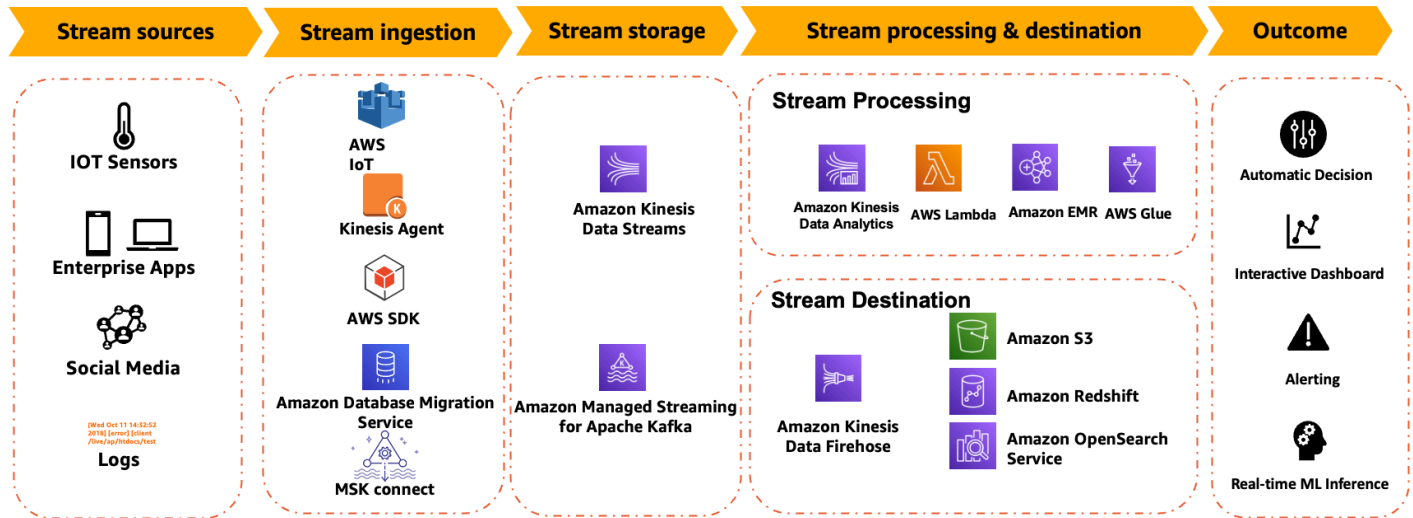


Modern data streaming architecture on AWS

The modern data streaming architecture includes the following key components:

- **Source** - Your source of streaming data includes data sources like sensors, social media, IoT devices, log files generated by using your web and mobile applications, mobile devices that generates semi-structured and unstructured data as continuous streams at high velocity.
- **Stream storage** - The stream storage layer is responsible for providing scalable and cost-effective components to store streaming data. The streaming data can be stored in the order it was received for a set duration of time, and can be replayed indefinitely during that time.
- **Stream ingestion** - The stream ingestion layer is responsible for ingesting data into the stream storage layer. It provides the ability to collect data from tens of thousands of data sources and ingest in near real-time. .
- **Stream processing** - The stream processing layer is responsible for transforming data into a consumable state through data validation, cleanup, normalization, transformation, and enrichment. The streaming records are read in the order they are produced, allowing for real-time analytics, building event driven applications, or streaming ETL.
- **Destination** - The destination layer is like a purpose-built destination depending upon your use case. Your destination can be an event driven application, data lake, data warehouse, database, or an OpenSearch.

Refer to the following diagram for an example of the streaming data lifecycle on AWS:

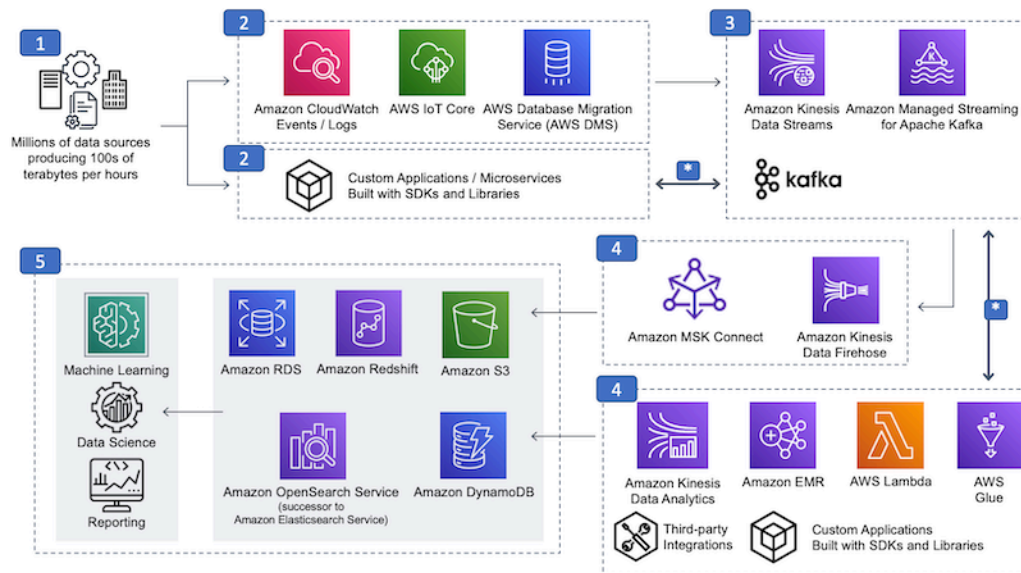


Streaming data lifecycle on AWS

The streaming data lifecycle is segmented into the layers of modern data streaming architecture:

- **Stream sources** - Streaming data sources can be application and click stream logs, mobile apps, existing transactional relational and NoSQL databases, IoT sensors and social media.
- **Stream ingestion** - [AWS IoT](#) for ingesting IoT devices data into Kinesis Data Streams and Amazon MSK, [Kinesis Agent](#) for ingesting streaming data into Kinesis Data Streams and Amazon Data Firehose, [AWS SDK](#) for custom producers, [AWS DMS](#) for change data capture use cases and Amazon MSK connect for continuous ingest from files, change data capture from databases.
- **Stream storage** - You can use Kinesis Data Streams, Amazon MSK and [Apache Kafka](#) on Amazon EC2 for your stream storage depending upon your use case. See Table 2 for additional details.
- **Stream processing** - You can use Managed Service for Apache Flink for advanced streaming use cases with multiple destinations and stateful stream processing. [AWS Lambda](#) is good for event-based and stateless processing, and use cases like filtering, enrichments, and transformations. Use [Amazon EMR](#) to use your favorite open-source big data frameworks and use [AWS Glue](#) if you are already using AWS Glue or Apache Spark where you need to process data in batch, streaming, and event modes and you want to build your streaming jobs visually.
- **Downstream destinations** - Your destination can be databases, data warehouses, purpose-built systems such as OpenSearch services, data lakes, event driven applications, and various third-party integrations.

This diagram represents the modern streaming reference architecture on AWS.



Streaming reference architecture on AWS

For more details about this architecture, refer to [Streaming reference architecture](#) in the AWS Well-Architected Data Analytics Lens.

Working with streaming data on AWS

Customers want the freedom to move data between their centralized data lakes and the surrounding purpose-built data services in a seamless, secure, and compliant way, to get insights with speed and agility.

For example, many organizations store streaming data in a data lake for offline analytics, and a portion of that data lake data can be moved out to a data warehouse for daily reporting. Think of this concept as *inside-out data movement*.

You can also move data in the other direction: from the outside-in. For example, you can move streaming data from non-relational databases into the data lake for product recommendation by using ML algorithms. Think of this concept as *outside-in data movement*.

In other situations, you may want to move data from one purpose-built data store to another. For example, you may copy the product catalog data stored in your database to your search service to make it easier to look through your product catalog, and offload the search queries from the database. Think of this concept as data movement *around the perimeter*.

The volume of data produced is increasing rapidly, and the data is coming from a wide variety of sources, in a variety of forms. The data is coming at lightning speeds due to an explosive growth of real-time data sources. Organizations create value by making decisions from their data. The faster they can make decisions and take action, the better they perform against their competitors. Yet, the value of data diminishes over time. To get the most value from the data, it must be processed at the velocity in which it is created at the source. Therefore, organizations need to work with the real-time data to deliver a better customer experience and to improve customer engagement.

Streaming data includes a wide variety of data, such as log files generated by customers using your mobile or web applications, ecommerce purchases, in-game player activity, information from social networks, financial trading floors, geospatial services, and telemetry from connected devices or instrumentation in data centers.

For example, sensors in transportation vehicles, industrial equipment, and farm machinery send data to a streaming application. The application monitors performance, detects any potential defects in advance, and places a spare part order automatically preventing equipment down time.

Organizations are also building real-time data streaming workloads to unlock the value of low-latency insights by moving from queue to a pub/sub model for a centralized messaging platform,

building asynchronous integrations with streaming data services, real-time device and fleet monitoring, application modernization (moving from monolith to microservices), real-time clickstream analytics, and streaming extract, transform, and load (ETL), anomaly and fraud detection, tailoring customer experience in real time, empowering IoT analytics and real-time personalization.

AWS provides several options to work with streaming data. You can take advantage of the managed streaming data services offered by Amazon Kinesis, Amazon MSK, [Amazon EMR Spark streaming](#), or deploy and manage your own streaming data solution in the cloud on [Amazon Elastic Compute Cloud](#) (Amazon EC2).

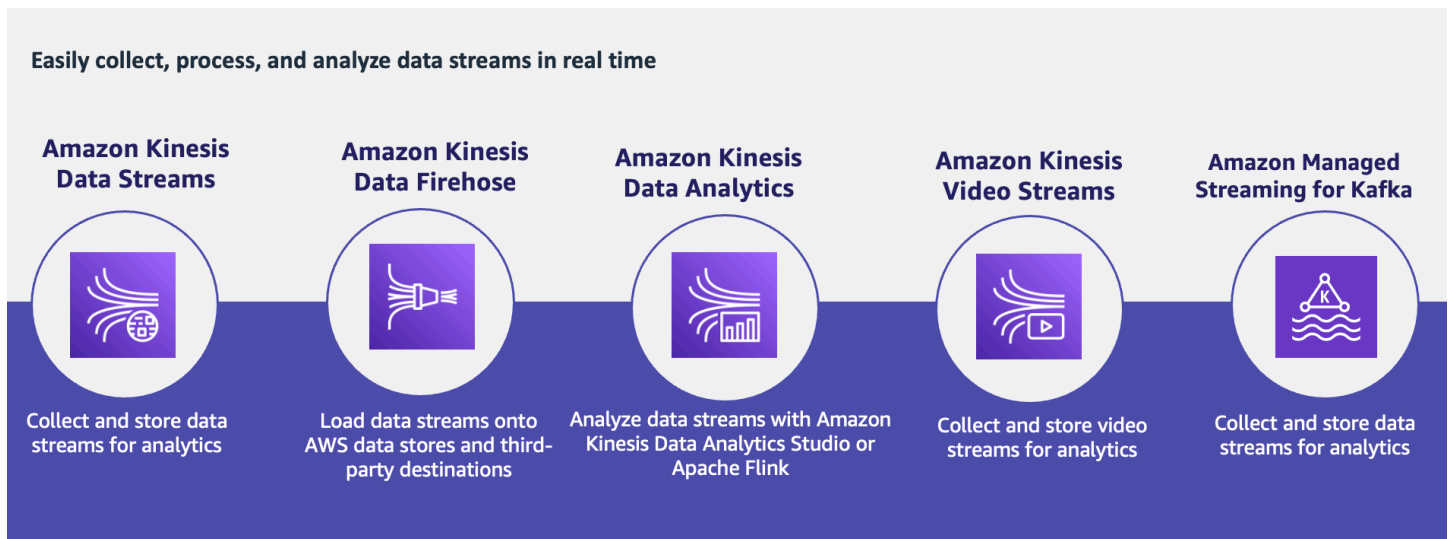
[Kinesis](#) is a platform for streaming data on AWS, offering powerful services that make it simple to load and analyze streaming data. It also enables you to build custom streaming data applications for specialized needs. If you have a streaming use case and you want to use an AWS native, fully managed service, consider Amazon Kinesis. It offers four services:

- [Amazon Kinesis Data Streams](#) - Collect and store data streams with Kinesis Data Streams, a scalable and durable real-time data streaming service that can continuously capture gigabytes of data per second from hundreds of thousands of sources.
- [Amazon Data Firehose](#) - Capture, transform, and load data streams into AWS data stores for near-real-time analytics with existing business intelligence tools.
- [Amazon Managed Service for Apache Flink](#) - Process and analyze data streams in real time with SQL or Apache Flink without having to learn new programming languages or processing frameworks.
- [Amazon Kinesis Video Streams](#) - Collect and store video streams with Kinesis Video Streams, which makes it simple to securely stream video from connected devices to AWS for analytics, ML, and other processing jobs.

Apache Kafka has been around for over ten years and tens of thousands of customers have been using Kafka to ingest streaming data. To enhance and reduce the overhead of managing Apache Kafka, AWS has introduced [Amazon MSK](#). If open-source technology is critical for your data processing strategy, you're familiar with Apache Kafka and you're looking for real-time latency in less than 70 milliseconds, AWS recommends Amazon MSK rather than Amazon Kinesis.

In addition, you can run other streaming data platforms such as Apache Flume, Apache Spark Streaming, and Apache Storm on Amazon EC2 and Amazon EMR.

The following diagram illustrates the various streaming services available on AWS.



Real-time streaming on AWS

Amazon Kinesis Data Streams

[Amazon Kinesis Data Streams](#) enables you to build your own custom applications that process or analyze streaming data for specialized needs. It can continuously capture and store terabytes of data per day from hundreds of thousands of sources. You can then build applications that consume the data from Kinesis Data Streams to power near real-time dashboards, generate alerts, implement dynamic pricing and advertising, and more. Kinesis Data Streams supports your choice of stream processing framework including Kinesis Client Library (KCL), Apache Storm, and Apache Spark Streaming.

With Kinesis Data Streams, you can ingest [real-time data such as application logs](#), website clickstreams, and Internet of Things (IoT) telemetry data for ML, analytics, and other applications. In addition to streaming ingestion use cases, you can also use Kinesis Data Streams to build applications for high-frequency event data such as clickstream data, and gain access to insights in seconds using AWS Lambda or Amazon Managed Service for Apache Flink. You can also use Kinesis Data Streams to power event-driven applications by quickly pairing with AWS Lambda to respond to or adjust immediate occurrences within the event-driven applications in your environment.

As organizations adopt data streaming more broadly, workloads with data traffic that can increase by millions of events in a few minutes are becoming more common. For these volatile traffic patterns, organizations carefully plan capacity, monitor throughput, and in some cases develop processes that automatically change the data stream capacity. [Kinesis Data Streams On-Demand](#) is a new capacity mode that eliminates the need for provisioning and managing the capacity for

streaming data. Kinesis Data Streams On-Demand automatically scales the capacity in response to varying data traffic. You're charged per gigabyte of data written, read, and stored in the stream, in a pay-per-throughput fashion.

Amazon Kinesis Data Streams services offer integration with modern data architecture in following ways to unlock new value from your data, such as improving operational efficiency, optimizing processes, developing new products and revenue streams, and building better customer user experiences.

- You can use [AWS Database Migration Service](#) (AWS DMS) to capture real-time transactions from relational databases and push data to an Amazon Kinesis data stream.
- You can use Amazon Kinesis Data Streams to capture changes to Amazon DynamoDB. Kinesis Data Streams captures item-level modifications in any DynamoDB table and replicates them to a [Kinesis data stream](#). Your applications can access this stream and view item-level changes in near real-time.
- You can create AWS Glue streaming ETL jobs that run nearly continuously and consume data from Kinesis Data Streams. This job cleans and transforms the data, then loads the results into Amazon S3 data lakes or Java Database Connectivity (JDBC) data stores.
- Kinesis Data Streams has integration with databases such as [Amazon Relational Database Service](#) (Amazon RDS) and [Amazon Aurora](#). For example, you can [stream Amazon RDS database changes into Kinesis Data Streams](#) for analytics. You can also push [Aurora DB cluster activities to a Kinesis data stream](#) and then configure other AWS services such as Firehose and Lambda to consume the stream and store the data.
- Amazon Redshift has launched [streaming ingestion support](#) for Kinesis Data Streams. Amazon Redshift streaming ingestion eliminates the need to stage data in Amazon S3 before ingesting it into Amazon Redshift, enabling you to achieve low latency in seconds while ingesting hundreds of megabytes of streaming data per second into your data warehouse.
- You can also create a stream in [Amazon Quantum Ledger Database \(Amazon QLDB\)](#) that [captures every document revision](#) that is committed to your journal and delivers this data to Kinesis Data Streams. A QLDB QLDB stream is a continuous flow of data from your ledger's journal to a Kinesis data stream resource. Then you use the Kinesis streaming platform or Kinesis Client Library to consume your stream, process the data records, and analyze the data contents.
- You can [send AWS API call events](#) in [Amazon EventBridge](#) to a Kinesis data stream, create Kinesis Data Streams applications, and process large amounts of data.
- Kinesis Data Streams and [Amazon CloudWatch](#) are [integrated](#) so you can collect, view, and analyze CloudWatch metrics for your Kinesis data streams. For example, to track shard usage,

you can monitor the *IncomingBytes* and *OutgoingBytes* metrics and compare them to the number of shards in the stream.

- An [Amazon API Gateway](#) REST API can act as a proxy to Kinesis Data Streams, adding either an individual data record or a list of data records.
- Kinesis Data Streams has integrations with other services like [AWS IoT](#), [Amazon CloudFront](#), [Connect Customer](#), and [AWS Lambda](#) for building low-latency streaming applications.

Amazon Data Firehose

Amazon Data Firehose is the easiest way to load streaming data into AWS. It can capture, transform, and deliver streaming data to Amazon S3, Amazon Redshift, OpenSearch Service, generic HTTP endpoints, and service providers such as Datadog, New Relic, MongoDB, and Splunk.

Firehose is a fully managed service that automatically scales to match the throughput of your data. It transforms and processes data on the fly with its built-in data transformation features with no code, no servers, and no ongoing maintenance from the customer. It can also batch, compress, transform, and encrypt your data streams before loading, minimizing the amount of storage used and increasing security.

Our customers are using Amazon Data Firehose for various use cases:

- Capture data continuously from connected devices such as consumer appliances, embedded sensors, and TV set-top boxes. Firehose loads the data into your specified destinations, enabling near real-time access to metrics, insights, and dashboards.
- Detect application errors as they happen and identify root cause by collecting, monitoring, and analyzing log data. You can easily install and configure the [Amazon Kinesis Agent](#) on your servers to automatically watch application and server log files and send the data to Firehose. Firehose continuously streams the log data to your destinations so you can visualize and analyze the data.
- Perform real-time analytics on data that has been traditionally analyzed using batch processing. Common streaming use cases include sharing data between different applications, streaming ETL, and real-time analytics. For example, you can use Firehose to continuously load streaming data into your Amazon S3 data lake or analytics services.
- Ingest [near real-time clickstream data](#), enabling marketers to connect with their customers in the most effective way. You can stream billions of small messages that are compressed, encrypted, and delivered to your destinations. From there, you can aggregate, filter, and process the data, and refresh content performance dashboards in near real-time.

Firehose offers integration with modern data architecture in the following ways to derive new and deeper insights from your data.

- Firehose can capture, transform, and load streaming data into Amazon S3, enabling near real-time analytics (as an outside-in data movement approach).
- You can also use Firehose to automatically convert the incoming data to open and standard-based formats like Apache Parquet and Apache ORC before the data is delivered (as an inside-out data movement approach).
- Columnar formats like Apache Parquet and ORC help to optimize the queries, reduce storage space needs and save costs.
- Firehose dynamic partitioning enables you to continuously partition streaming data in Firehose by using keys within data (for example, *customer_id* or *transaction_id*) and then deliver the data grouped by these keys into corresponding Amazon S3 prefixes. This makes it easier to run high-performance, cost-efficient analytics on streaming data in Amazon S3 using various services such as Athena, Amazon EMR, Amazon Redshift Spectrum, and Quick. In addition, AWS Glue can perform more sophisticated ETL jobs after the dynamically partitioned streaming data is delivered to Amazon S3, in use cases where additional processing is required.

Amazon Managed Service for Apache Flink

[Amazon Managed Service for Apache Flink](#) is the easiest way to transform and analyze streaming data in real-time with [Apache Flink](#). Apache Flink is an open-source framework and engine for processing data streams. Managed Service for Apache Flink reduces the complexity of building, managing, and integrating Apache Flink applications with other AWS services.

Managed Service for Apache Flink takes care of everything required to [run streaming applications nearly continuously](#), and scales automatically to match the volume and throughput of your incoming data. With Managed Service for Apache Flink, there are no servers to manage, no minimum fee or setup cost, and you only pay for the resources your streaming applications consume.

Managed Service for Apache Flink has the following integration with other AWS services for seamless data movement.

- Develop streaming ETL applications with Managed Service for Apache Flink built-in operators to transform, aggregate, and filter streaming data. You can easily deliver your data in seconds to

Amazon Kinesis Data Streams, Amazon MSK, Amazon OpenSearch Service, Amazon S3, custom integrations, and more using built-in connectors.

- Analyze streaming data interactively with Managed Service for Apache Flink Studio and build using your preferred languages, tools, and developer environments. With Managed Service for Apache Flink Studio, you can interactively query data streams using your language of choice and view results in seconds.
- Develop applications that process events from one or more data streams and trigger conditional processing and external actions. You can use Apache Flink libraries for complex event processing and then, store proceed event into data lake for offline analysis.

Amazon Kinesis Video Streams

[Amazon Kinesis Video Streams](#) makes it simple to securely stream video from connected devices to AWS for analytics, ML, and other processing.

It automatically provisions and elastically scales all the infrastructure needed to ingest streaming video data from millions of devices. It durably stores, encrypts, and indexes video data in your streams, and allows you to access your data through easy-to-use APIs.

Kinesis Video Streams enables you to play back video for live and on-demand viewing, and quickly build applications that take advantage of computer vision and video analytics through integration with [Amazon Rekognition Video](#), and libraries for ML frameworks such as Apache MXNet, TensorFlow, and OpenCV.

Kinesis Video Streams has the following integration with AWS Lake House services for seamless data movement:

- It uses Amazon S3 as the underlying data store, which means your data is stored durably and reliably. You can quickly search and retrieve video fragments based on device- and service-generated timestamps.
- You can easily build applications with real-time computer vision capabilities and real-time video analytics capabilities using popular open-source ML frameworks.

Amazon MSK

[Amazon Managed Streaming for Apache Kafka](#) (Amazon MSK) is a fully managed service that makes it simple for you to build and run applications that use Apache Kafka to process streaming

data. Apache Kafka is an open-source platform for building real-time streaming data pipelines and applications. With Amazon MSK, you can use native Apache Kafka APIs to populate data lakes, stream changes to and from databases, and power ML and analytics applications. You can also use the [AWS Glue Schema Registry](#) to validate and control the evolution of schemas used by Apache Kafka applications.

Apache Kafka clusters are challenging to set up, scale, and manage in production. When you run Apache Kafka on your own, you need to provision servers, configure Apache Kafka manually, replace servers when they fail, orchestrate server patches and upgrades, architect the cluster for high availability, ensure data is durably stored and secured, setup monitoring and alarms, and carefully plan scaling events to support load changes. [Amazon MSK makes it easy for you to build and run production applications on Apache Kafka](#) without needing Apache Kafka infrastructure management expertise. That means you spend less time managing infrastructure and more time building applications.

[Amazon MSK Serverless](#) is a cluster type for Amazon MSK that enables you to run Apache Kafka without having to manage and scale cluster capacity. Amazon MSK Serverless automatically provisions and scales compute and storage resources, so you can use Apache Kafka on demand and pay for the data you stream and retain.

Amazon MSK has the following integration with AWS modern data architecture for seamless data movement, to build purpose-built analytics from your data:

- Amazon MSK integrates with Lambda and Managed Service for Apache Flink for Apache Flink applications and the Amazon EMR Spark streaming applications to process streaming data in near real-time using the inside-out data movement approach.
- Amazon MSK as an event source operates similarly to using Amazon Simple Queue Service (Amazon SQS) or Kinesis. Lambda internally polls for new records or messages from the event source, and then synchronously invokes the target Lambda function. Lambda reads the messages in batches and provides these to your function as an event payload. It continues to process batches until there are no more messages in the topic.
- Amazon MSK integrates [AWS IoT](#) for IoT event sourcing using IoT rule action to deliver messages from your devices directly to your Amazon MSK. You can use this for data analysis and visualization, without writing a single line of code as the outside-in data movement approach.
- AWS DMS can capture data from online transaction processing (OLTP) database systems, and push data to Amazon MSK as producer using the outside-in data movement approach.

- Amazon MSK also offers Amazon MSK Connect, a managed Kafka Connect offering that you can use to move data from a wide variety of data sources and sinks. It can also be used for transformations using Single Message Transform (SMT) or develop custom logic.

You can integrate Amazon MSK with Firehose using a Lambda function that processes process records in a Kafka topic and deliver it to a Firehose delivery stream, which buffers data before delivering it to the destination such as an Amazon S3 bucket that stores all events from the Amazon MSK cluster for offline analysis.

Streaming architecture patterns using a modern data architecture

Organizations perform streaming analytics to build better customer experiences in near real-time to stay ahead of their competitors because the value of data diminishes over time. To be near real time, data needs to be produced, captured, and processed with low latency. Organizations need a system that scales to support the modern data architecture needs, but also allows them to build their own applications on top of the data collected. The order is critical, because applications need to be able to tell the story of what happened, when it happened, and how it happened, relative to other events in the pipeline.

The modern data architecture on AWS provides a strategic vision of how multiple AWS data and analytics services can be combined into a multi-purpose data processing and analytics environment to address these challenges.

Low latency modern data streaming applications

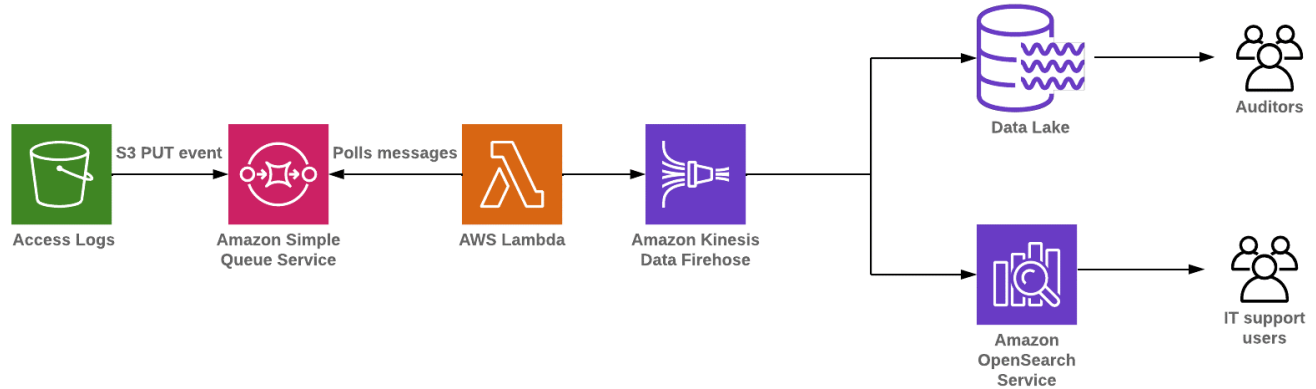
The following are a few use cases for when you need to move data around your purpose-built data services with low latency for faster insights, and how to build these streaming application architectures with AWS streaming technologies.

Build access logs streaming applications using Firehose and Managed Service for Apache Flink

Customers perform log analysis that involves searching, analyzing, and visualizing machine data generated by their IT systems and technology infrastructure. It includes logs and metrics such as user transactions, customer behavior, sensor activity, machine behavior, and security threats. This data is complex, but also the most valuable because it contains operational intelligence for IT, security, and business.

In this use case, customers have collected log data in an Amazon S3 data lake. We need to access log data and analyze it in a variety of ways, using the right tool for the job for various security and compliance requirements. There are several data consumers including auditors, streaming analytics users, enterprise data warehouse users, and so on. They need to keep a copy of the data for regulatory purposes.

The following diagram illustrates the modern data architecture with access logs data as an input to derive near real-time dashboards and notifications.



Access logs streaming applications for anomaly detection using Amazon Managed Service for Apache Flink and Amazon OpenSearch Service

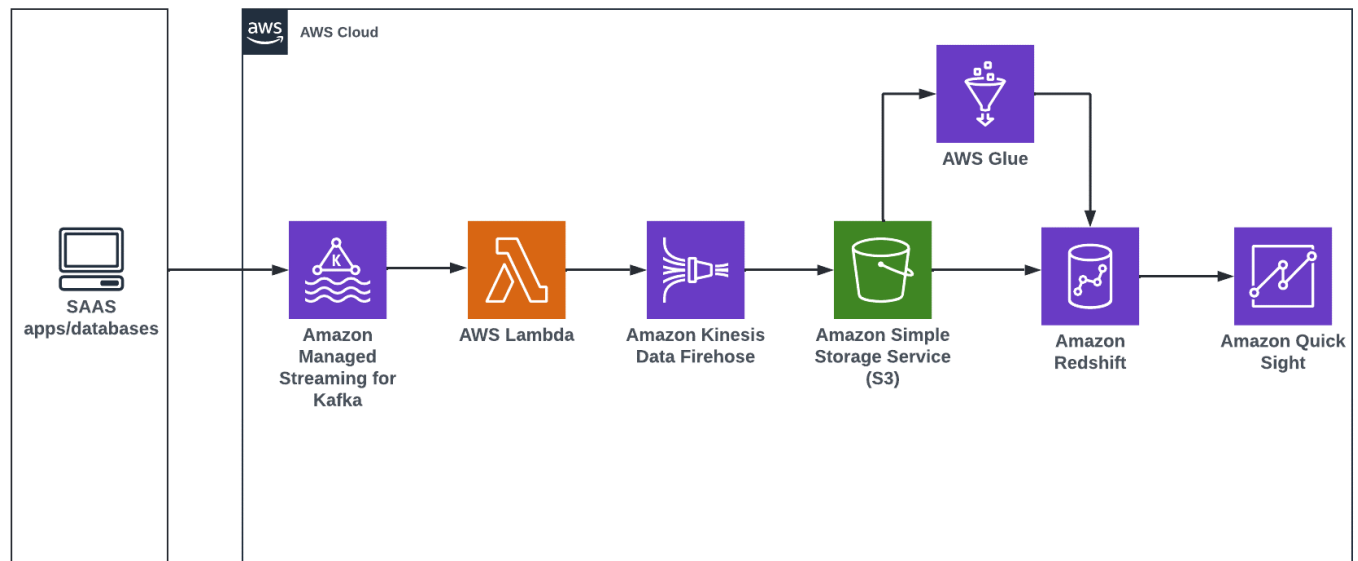
The steps that follow the architecture are:

1. Logs from multiple sources such as [Amazon CloudFront](#) access logs, VPC Flow Logs, API logs, and application logs are pushed into the data lake.
2. Amazon S3 PUT events are published to Amazon SQS events. AWS Lambda polls the events from Amazon SQS and invokes a Lambda function to move data into multiple sources such as Amazon S3 and Amazon OpenSearch Service.
3. You can build low latency modern data streaming applications by creating a near real-time OpenSearch dashboard.
4. You can also store access log data into Amazon S3 for archival, and load sub-access log summary data into Amazon Redshift, depending on your use case.

Stream data from diverse source systems into the data lake using MSK for near real-time reports

Customers want to stream near real-time data from diverse source systems such as Software as a Service (SaaS) applications, databases, and social media into Amazon S3, and to online analytical processing (OLAP) systems such as Amazon Redshift, to derive user behavior insights and to build better customer experiences. Hopefully, this will drive customers toward more reactive, intelligent, near real-time experiences. You can use this data in Amazon Redshift to develop customer-centric business reports to improve overall customer experience.

The following diagram illustrates the modern data architecture with input stream data to derive near real-time dashboards.



Derive insights from input data coming from diverse source systems for near real-time dashboards with Quick.

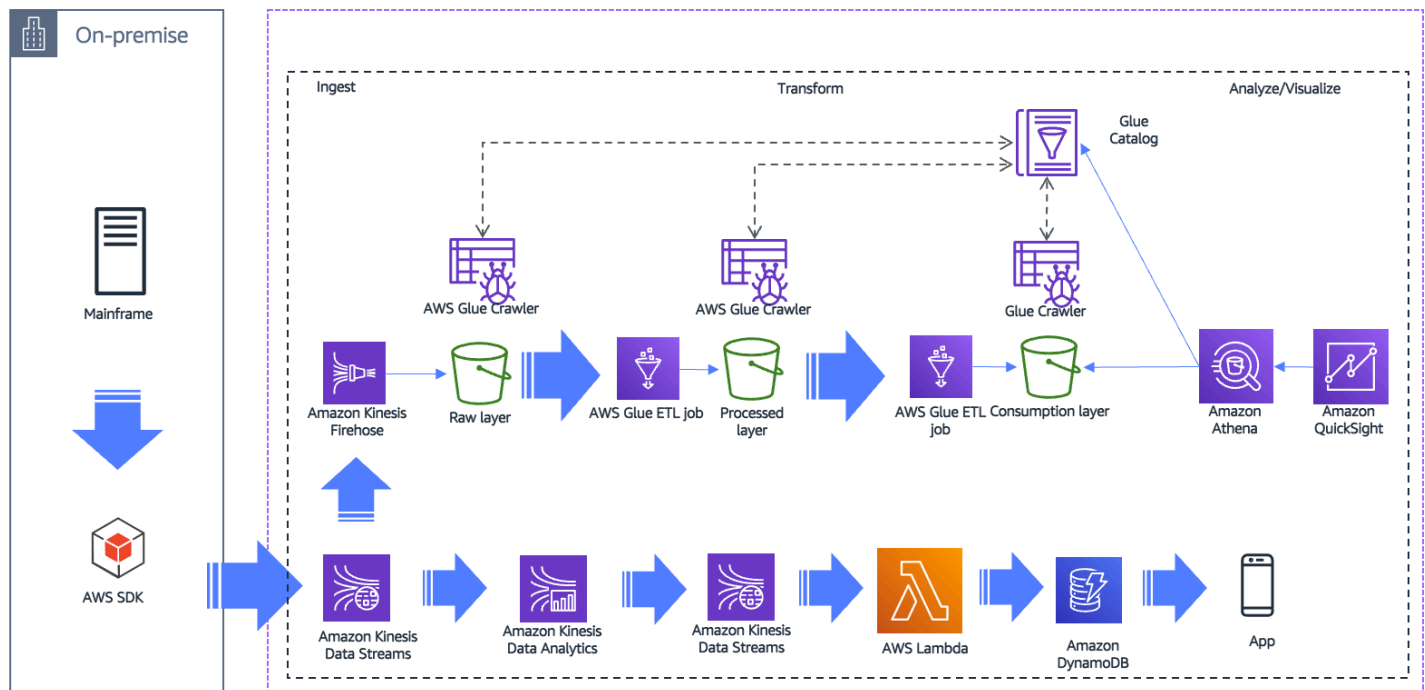
The steps that follow the architecture are:

1. You can stream near real-time data from source systems such as social media using Amazon MSK, Lambda, and Firehose into Amazon S3.
2. You can use AWS Glue for data processing, and load transformed data into Amazon Redshift using a Glue development endpoint such as [Amazon SageMaker AI notebook instances](#).
3. When data is in Amazon Redshift, you can create a customer-centric business report using Quick.

Build a serverless streaming data pipeline using Amazon Kinesis and AWS Glue

Customers want low-latency near real-time analytics to process user behavior and respond almost instantaneously with relevant offers and recommendations. The customer's attention will be lost if these recommendations are not available for days, hours, or even minutes – they need to happen in near real-time. The following diagram illustrates a typical modern data architecture for a streaming

data pipeline to keep the application up to date, and to store streaming data into a data lake for offline analysis.



Build a serverless streaming data pipeline

The steps that follow the architecture are:

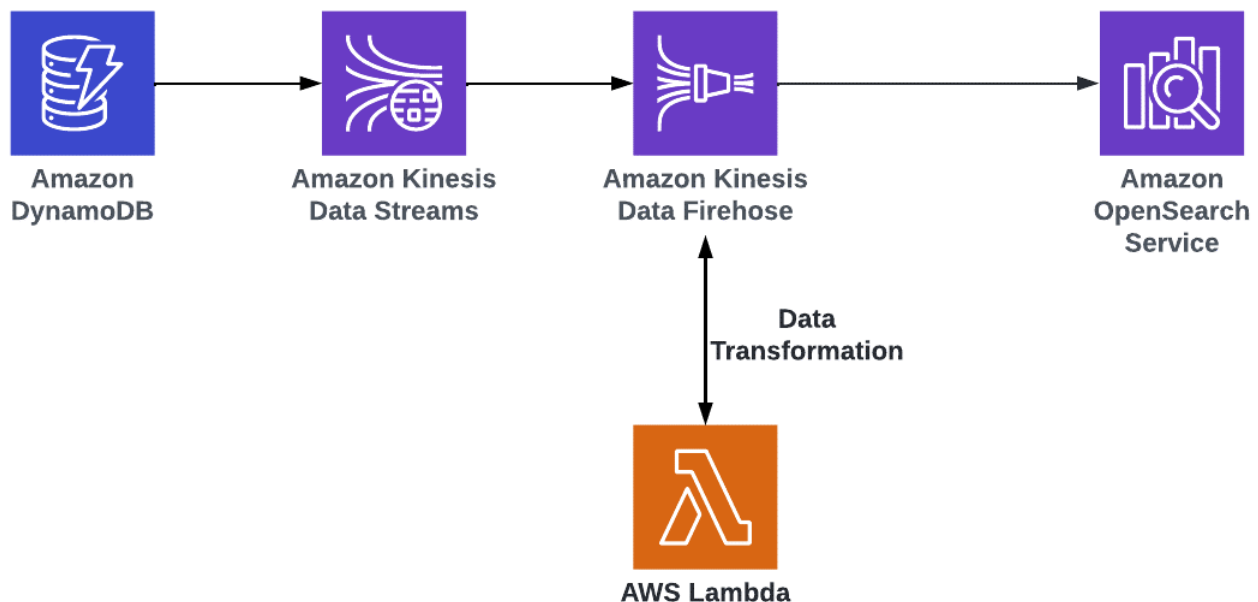
1. Extract data in near real-time from an on-premises legacy system to a streaming platform such as Apache Kafka. From Kafka, you can move the data to Kinesis Data Streams.
2. Use Managed Service for Apache Flink to analyze streaming data, gain actionable insights, and respond to your business and customer needs in near real-time.
3. Store analyzed data in cloud scale databases such as Amazon DynamoDB, and push to your end users in near real-time.
4. Kinesis Data Streams can use Firehose to send the same streaming content to the data lake for non-real-time analytics use cases.
5. AWS Glue includes an ETL engine that you can use to transform data. AWS Glue crawlers automatically update the metadata store as new data is ingested.
6. Analytics teams use data in the consumption layer with analytics solutions such as Athena and Quick.

Set up near real-time search on DynamoDB table using Kinesis Data Streams and OpenSearch Service

Organizations want to build a search service for their customers to find the right product, service, document, or answer to their problem as quickly as possible. Their searches will be across both semi-structured and unstructured data, and across different facets and attributes. Search results have to be relevant and delivered in near real-time. For example, if you have an ecommerce platform, you want customers to quickly find the product they're looking for.

You can use both DynamoDB and OpenSearch Service to build a near real-time search service. You can use DynamoDB as a durable store, and OpenSearch Service to extend its search capabilities. When you set up your DynamoDB tables and streams to replicate your data into OpenSearch Service, you can perform near real-time, full-text search on your data.

The following diagram illustrates the modern data architecture with Amazon DynamoDB and Amazon OpenSearch Service.



Derive insights from Amazon DynamoDB data by setting up near real-time search using Amazon OpenSearch Service

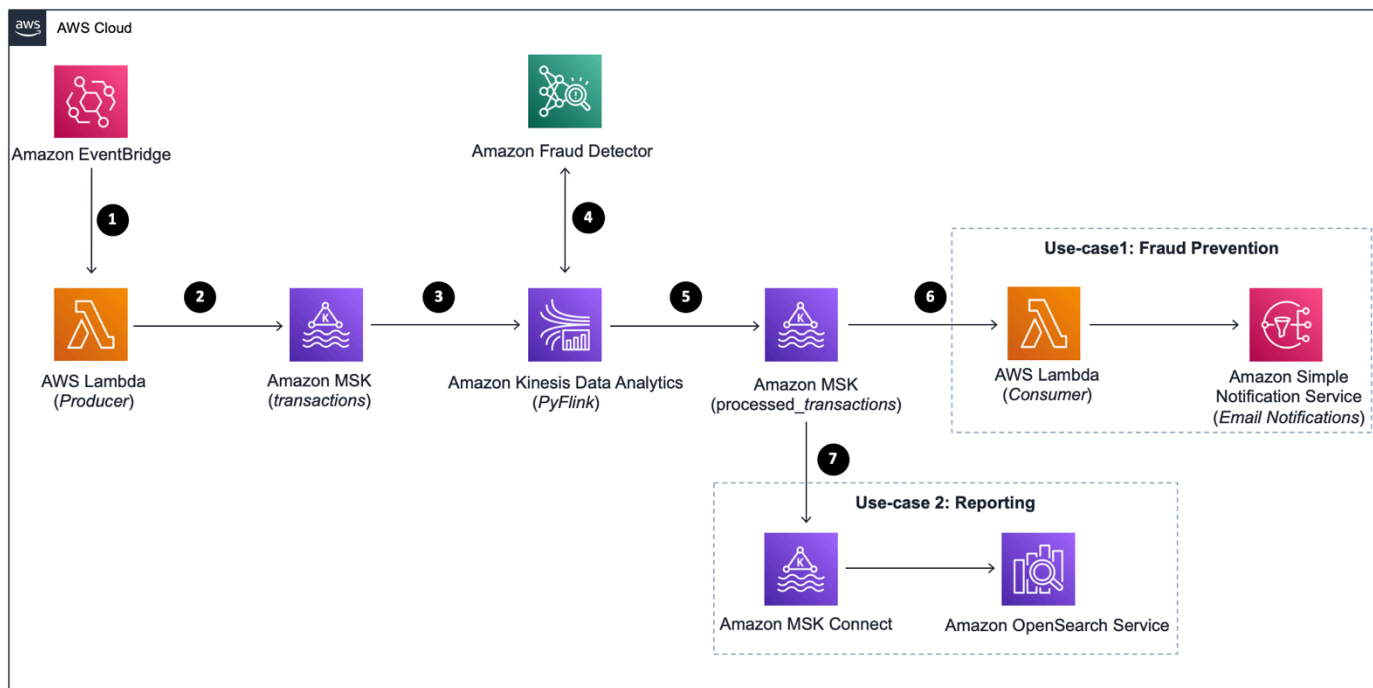
The steps that follow the architecture are:

1. In this design, the DynamoDB table is used as the primary data store. An [Amazon OpenSearch Service](#) cluster is used to serve all types of searches by indexing the table.
2. DynamoDB has an integration with Kinesis Data Streams for change data capture (CDC) any update, deletion, or new item on the main table is captured and processed using AWS Lambda. Lambda makes appropriate calls to OpenSearch Service to index the data in near real-time.
3. For more details about this architecture, refer to [Indexing Amazon DynamoDB Content with Amazon OpenSearch Service Using AWS Lambda](#) and [Loading Streaming Data into Amazon OpenSearch Service from Amazon DynamoDB](#).
4. You can use the streaming functionality to send the changes to OpenSearch Service or Amazon Redshift by using a Data Firehose delivery stream. Before you load data into OpenSearch Service, you might need to transform the data. You can use Lambda functions to perform this task. For more information, refer to [Amazon Data Firehose Data Transformation](#).

Build a real-time fraud prevention system using Amazon MSK and Amazon Fraud Detector

Organizations with online businesses have to be on guard constantly for fraudulent activity such as fake accounts or payments made with stolen credit cards. One way they try to identify fraudsters is by using fraud detection applications. Emerging technologies like artificial intelligence (AI) and machine learning (ML) can provide a solution that shifts from enforcing rule-based validations to using validations based on learning from examples and trends directly found in the transaction data by specifying the key features that might contribute to fraudulent behavior, such as customer-related information (card number, email, IP address, and location) and transaction-related information (time, amount, and currency).

The following diagram illustrates the modern streaming data architecture for building fraud prevention system using Amazon MSK, Amazon Managed Service for Apache Flink, and Amazon Fraud Detector.



Build a fraud detection system on AWS

The first use case demonstrates fraud prevention by identifying fraudulent transactions, flagging them to be blocked, and sending an alert notification. The second writes all transactions in real time to Amazon OpenSearch Service, which enables real-time transaction reporting using OpenSearch dashboards.

The steps that follow the architecture are:

1. Use [AWS Lambda](#) function as a real-time transactions producer that can be scheduled to run every minute using an [Amazon EventBridge](#) rule.
2. Each transaction is written into an input Amazon MSK topic called *transactions*.
3. Process the payments in real time by using Apache Flink [Table API](#), which allows intuitive processing using relational operators such as selection, filter, and join. You can use the PyFlink Table API running as a Kinesis data analytics application.
4. The Kinesis data analytics application calls the Amazon Fraud Detector [GetEventPrediction](#) API to get the predictions in real time.
5. The Kinesis data analytics application writes the output containing the transaction outcome (fraud prediction) into an output Amazon MSK topic called *processed_transactions*.

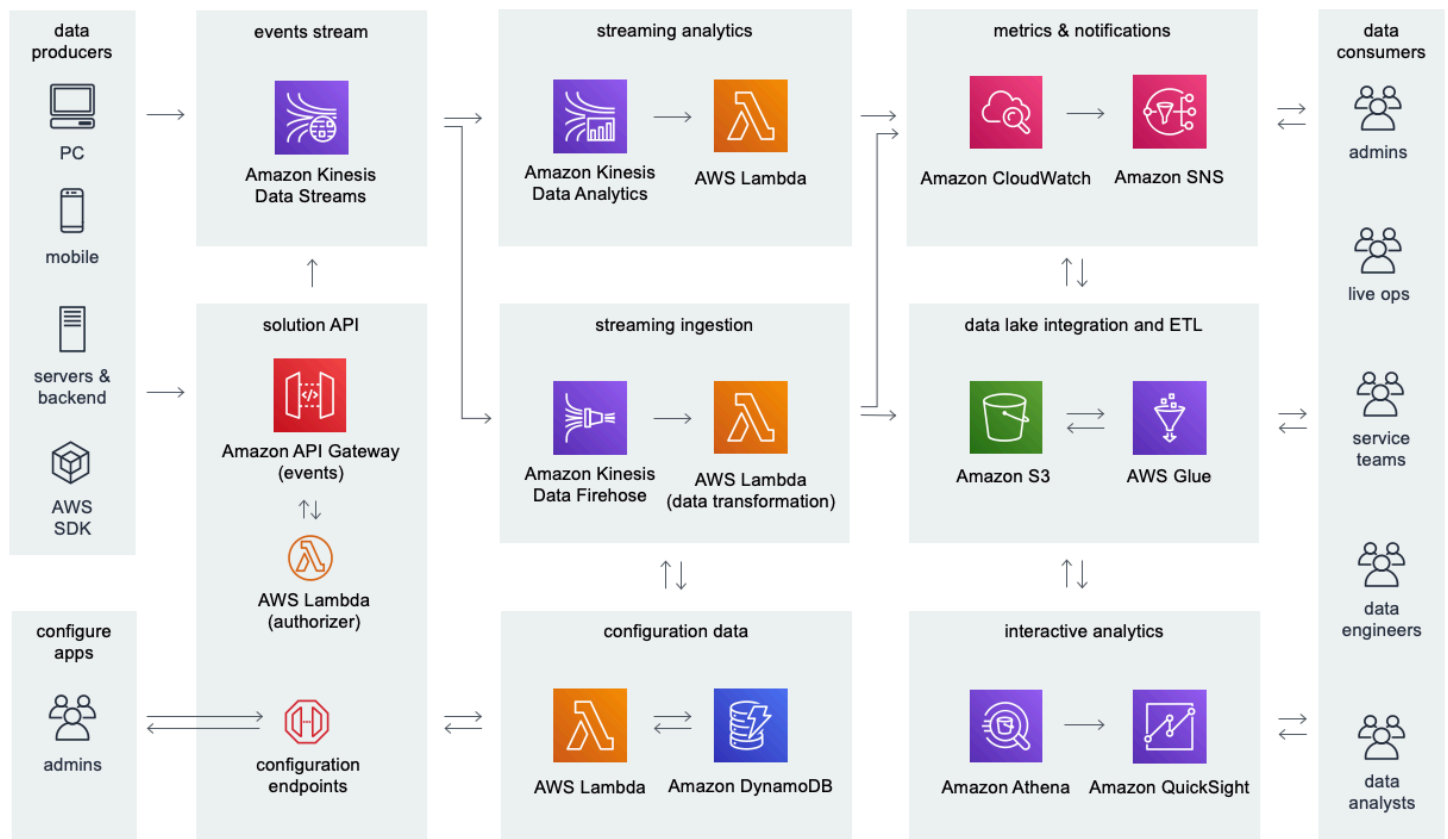
6. Use a AWS Lambda function to evaluate the outcome of each transaction. If the outcome is block, it generates an [Amazon Simple Notification Service](#) (Amazon SNS) notification to notify you by email.
7. Use Amazon MSK Connect to sink the data in real time to an OpenSearch Service visualization dashboard with fraud prediction results.

For more details about this architecture, refer to the [Build and visualize a real-time fraud prevention system](#) blog.

Stream games data from diverse source systems into a Lake using Kinesis Data Streams for real-time game insights

The Game Analytics Pipeline solution helps game developers launch a scalable serverless data pipeline to ingest, store, and analyze telemetry data generated from games and services. The solution supports streaming ingestion of data, allowing users to gain insights from their games and other applications within minutes.

The following diagram illustrates the modern data streaming architecture with streaming games data from various devices to derive real-time insights.



Derive real-time persona-centric games insights from various devices

The steps that follow the architecture are:

1. API Gateway provides REST API endpoints for registering game applications with the solution and for ingesting game telemetry data, which sends the events to Kinesis Data Streams. DynamoDB stores game application configurations and API keys.
2. You can send streaming game data to Kinesis Data Streams from your data producers, including game clients, game servers, and other applications, and enable real-time data processing by Firehose and Managed Service for Apache Flink. Firehose consumes the streaming data from Kinesis Data Streams and invokes Lambda with batches of events for serverless data processing and transformation before ingestion into Amazon S3 for storage.
3. AWS Glue provides ETL processing workflows and metadata storage in the AWS Glue Data Catalog, which provides the basis for a data lake for integration with flexible analytics tools. Sample Athena queries analyze game events, and integration with Quick is available for reporting and visualization. CloudWatch monitors, logs, and generates alarms for the utilization of AWS resources and creates an operational dashboard. Amazon Simple Notification Service (Amazon SNS) provides delivery of notifications to solution administrators and other data consumers when CloudWatch alarms are triggered.

Key considerations while building streaming analytics

When you are building a streaming data pipeline using modern data architecture to stream log and event data to power live dashboards and deliver data into data lakes, to build real-time analytics and event-driven applications and machine learning (ML), you must first understand the ideal usage patterns of AWS streaming data solutions, your user personas, and your specific use case so you can choose the right service for the job.

Choosing the right Kinesis service for your use case

The following table illustrates the ideal usage patterns of various Kinesis data streaming and processing services.

Table 1: Amazon Kinesis usage patterns

	Kinesis Data Streams	Firehose	Managed Service for Apache Flink
Usage	Capture stream log and event data, run real-time analytics, and build event-driven applications	Load data streams into AWS data stores	Analyze data streams with Managed Service for Apache Flink Studio and Apache Flink
Data sources	Mobile apps, application logs, web clickstream/social, IoT sensors, connected products, smart buildings	Connected devices such as consumer appliances, embedded sensors, TV set-top boxes, clickstream data, application logs	Analyze streaming data from Kinesis Data Streams, Amazon MSK, Amazon MQ , custom connectors
Stream ingestion	AWS SDKs , Kinesis Producer Library , AWS Mobile SDKs , Kinesis Agent , AWS IoT , Amazon	AWS SDKs, Kinesis Producer Library, Kinesis Data Streams, Kinesis Agent,	Analyze streaming data from Kinesis Data Streams, Amazon MSK,

	Kinesis Data Streams	Firehose	Managed Service for Apache Flink
	CloudWatch Events , Amazon DynamoDB , AWS DMS	AWS IoT, Amazon CloudWatch Events	Amazon MQ, custom connectors

Choosing the right streaming service for your use case

The following table illustrates the comparison between Apache Kafka, Kinesis Data Streams, and Amazon MSK.

Table 2 — Streaming services

Attribute	Apache Kafka	Kinesis Streams	MSK
Ease of use	Advanced setup required	Get started in minutes	Get started in minutes
Management Overhead	High	Low	Low (Amazon MSK Serverless) to Medium (Amazon MSK Provisioned)
Scalability	Difficult to scale	Scale in seconds with one click	Scale in minutes with one click
Throughput	Very large	Scale with Kinesis Data Streams on-demand	Very large
Infrastructure	You manage	AWS manages	AWS manages
Open-sourced?	Yes	No	Yes (managed service for Apache Kafka)
Data retention	You can retain data for longer duration, and it is configurable.	You can retain data for up to 365 days.	You can retain data for longer duration, and it is configura

Attribute	Apache Kafka	Kinesis Streams	MSK
			ble. With the tiered storage feature of Amazon MSK, you can cost-efficiently store vast amounts of data in Amazon S3.
Latency	Low	Low (70ms with Enhance Fan Out)	Lowest

Choosing the right streaming data processing technology

Streaming data processing technologies support many use cases that include event-driven applications, data analytics applications, and data pipeline applications. Commonly used frameworks include [Apache Kafka Streams](#), [Apache Flink](#), [KSQL](#), and Managed Service for Apache Flink for Flink. Apache Kafka Streams, [Apache Flink](#), and KSQL are open-source options, while Amazon Managed Service for Apache Flink offers a fully managed Apache Flink.

The following table illustrates the comparison between Apache Kafka Streams, Managed Service for Apache Flink for Apache Flink, and Managed Service for Apache Flink SQL.

Table 3 — Comparison between data stream processing technologies

Feature	Apache Kafka Streams	Managed Service for Apache Flink for Apache Flink	Kinesis Client Library	Lambda
Open source	Yes	Based on open-source Apache Flink	Based on Kinesis Client Library for Java open source	No, based on proprietary engine
Sources	Kafka only	Kinesis Data Streams,	Kinesis Data Streams	Kinesis Data Streams,

Feature	Apache Kafka Streams	Managed Service for Apache Flink for Apache Flink	Kinesis Client Library	Lambda
		Amazon MSK for Apache Kafka, DynamoDB, custom sources, RabbitMQ , RabbitMQ		Firehose, Amazon MSK
Destination/sinks	Kafka only; over 10 connectors supported with Kafka connect	Amazon MSK for Kafka, Kinesis Data Streams, Firehose, Amazon S3 Apache Cassandra, Amazon DynamoDB, OpenSearch Service, custom sinks supported by open-source Flink	Multi-stream processing	Use AWS Lambda to respond to or adjust immediate occurrences within the event-driven applications. AWS Lambda can read records from Kinesis Data Streams and invokes your function.

Feature	Apache Kafka Streams	Managed Service for Apache Flink for Apache Flink	Kinesis Client Library	Lambda
Development languages	Java and Scala	Java, Scala, SQL, and Python	Java; support for languages other than Java is provided using a multi-language interface called the <i>MultiLang Daemon</i>	Java, .NET Core, Go, PowerShell, Node.js#, Python, Ruby; it supports multiple languages through the use of Lambda runtimes

Feature	Apache Kafka Streams	Managed Service for Apache Flink for Apache Flink	Kinesis Client Library	Lambda
Development process	Develop on any integrated development environment (IDE) using Java or Scala. The application is separate from the Kafka broker and needs to be scaled independently.	Develop on any IDE and build a JAR file. Create a Managed Service for Apache Flink application and upload application JAR.	The Kinesis Client Library (KCL) is a Java library. KCL helps you consume and process data from a Kinesis data stream by taking care of many of the complex tasks, such as load balancing across multiple consumer application instances, responding to consumer application instance failures, checkpointing processed records, and reacting to resharding.	Develop on any IDE that is supported by respective programming language

Feature	Apache Kafka Streams	Managed Service for Apache Flink for Apache Flink	Kinesis Client Library	Lambda
Exactly once processing support	Yes	Yes	Not built in	Not built in
Per record processing latency	Sub-second	Sub-second	Seconds	Seconds
Batch support	No	Yes, supported by Flink	No	Yes, with Amazon EventBridge

Key benefits

Modern data streaming analytics architecture on AWS provides the following key benefits:

- Organizations can build scalable modern data streaming architectures on AWS that can handle explosive data growth and velocity.
- A comprehensive set of integrated tools enables every user equally.
- The modern data architecture enables better business agility. It analyzes data and helps organizations gain insights as events happen in real time. By using purpose-built analytics services in a modern data architecture approach, data consumers can create business value by experimenting fast and responding to changes quickly.
- Because a modern data architecture provides a ring of purpose-built data services, organizations have the flexibility to choose the right services for the right use cases.
- All personas are empowered to use the best-fit analytics services.
- You can build seamless low latency analytics architectures for near real-time personalization to tailor the customer experience.
- Simplified streaming ingestion and cleaning enables data engineers to build streaming applications faster.
- You can take advantage of unified low latency streaming analytics across operational databases, data warehouse, and data lake.
- The modern data architecture enables unified governance, which provides a secure, compliant, and auditable environment for data analytics workloads.
- Centralized management of fine-grained permissions empowers security officers.
- The modern data architecture provides the required durability, availability, and scalability for various data analytics workloads. It can help you process exabytes of data stored within your data lakes and deploy models on ML services to serve hundreds of billions of inference requests.

Conclusion

An AWS modern data architecture provides various AWS purpose-built and managed streaming analytics services to satisfy low-latency use cases. Each of these streaming analytics services has its own characteristics in terms of scalability, latency, cost, and supported data sources. The seamless data movement empowered by AWS modern data architecture allows your organization to gain full insights from your data using various data analytics services.

With streaming analytics applications built with AWS modern data architecture, organizations can get low latency insights quickly with near real-time analytics without worrying about the underlining infrastructures. The flexibility and extensibility of AWS modern data architecture makes it simple to support new use cases by using new analytics services as they become available.

Contributors

Contributors to this document include:

- Raghavarao Sodabathina, Enterprise Solutions Architect, Amazon Web Services
- Changbin Gong, Principal Solutions Architect, Amazon Web Services
- Niyati Upadhyay, Solutions Architect, Amazon Web Services
- Soujanya Konka, Solutions Architect, Amazon Web Services

Further reading

For detailed architectural patterns, refer to the following resources:

- [Modern data architecture on AWS](#)
- [Harness the power of your data with AWS Analytics](#) (blog post)
- [Derive Insights from AWS Modern Data](#) (AWS whitepaper)
- [Design a data mesh architecture using AWS Lake Formation and AWS Glue](#) (blog post)
- [AWS Cloud Data Ingestion Patterns and Practices](#) (AWS whitepaper)
- [Creating a source to Lakehouse data replication pipe using Apache Hudi, AWS Glue, AWS DMS, and Amazon Redshift](#) (blog post)
- [Best practices for consuming Amazon Kinesis Data Streams using AWS Lambda](#) (blog post)
- [Best practices for Managed Service for Apache Flink](#) from the [Amazon Managed Service for Apache Flink for SQL Applications Developer Guide](#)
- [Security Best Practices for Firehose](#) from the [Amazon Data Firehose Developer Guide](#)
- [Best practices from Delhivery on migrating from Apache Kafka to Amazon MSK](#) (blog post)

Document revisions

Date	Description
May 17, 2022	First publication
August 25, 2022	Updated document and diagrams, added new sections.

Notices

Customers are responsible for making their own independent assessment of the information in this document. This document: (a) is for informational purposes only, (b) represents current AWS product offerings and practices, which are subject to change without notice, and (c) does not create any commitments or assurances from AWS and its affiliates, suppliers or licensors. AWS products or services are provided “as is” without warranties, representations, or conditions of any kind, whether express or implied. The responsibilities and liabilities of AWS to its customers are controlled by AWS agreements, and this document is not part of, nor does it modify, any agreement between AWS and its customers.

© 2022 Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS Glossary

For the latest AWS terminology, see the [AWS glossary](#) in the *AWS Glossary Reference*.